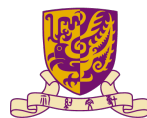


Network Programming RPC

Minchen Yu

SDS@CUHK-SZ

Spring 2026

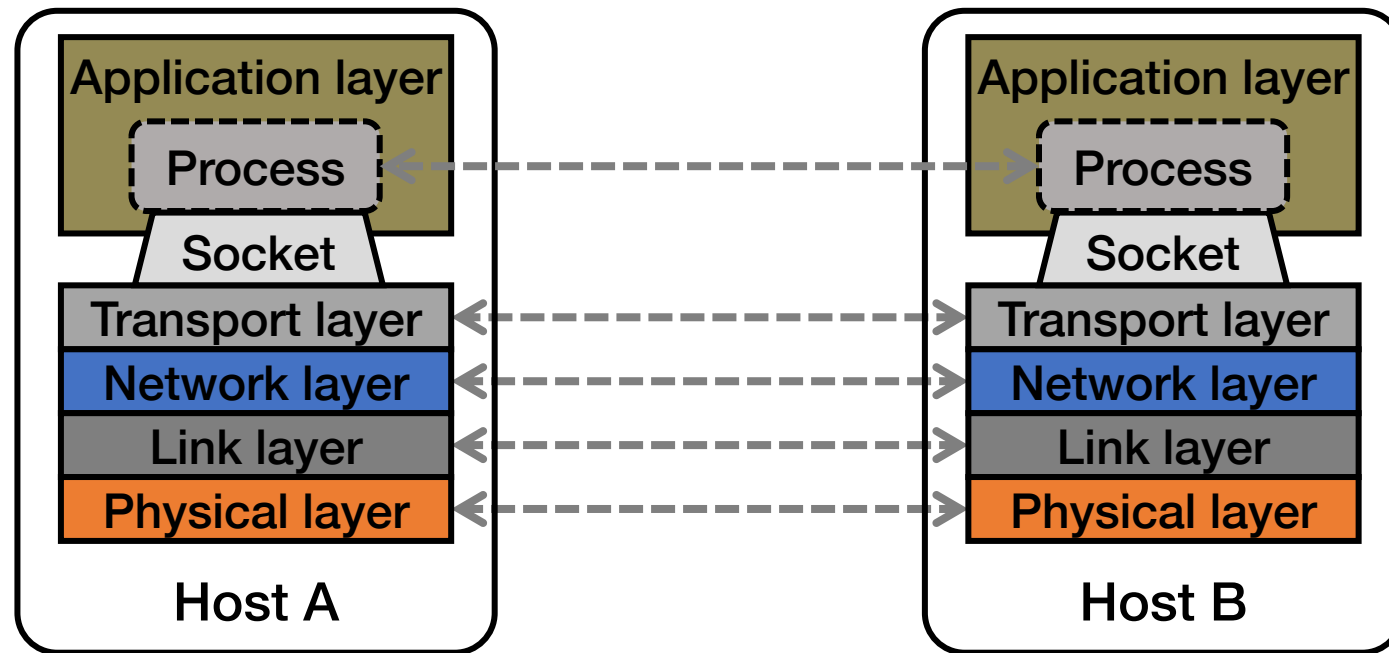


香港中文大學(深圳)
The Chinese University of Hong Kong, Shenzhen

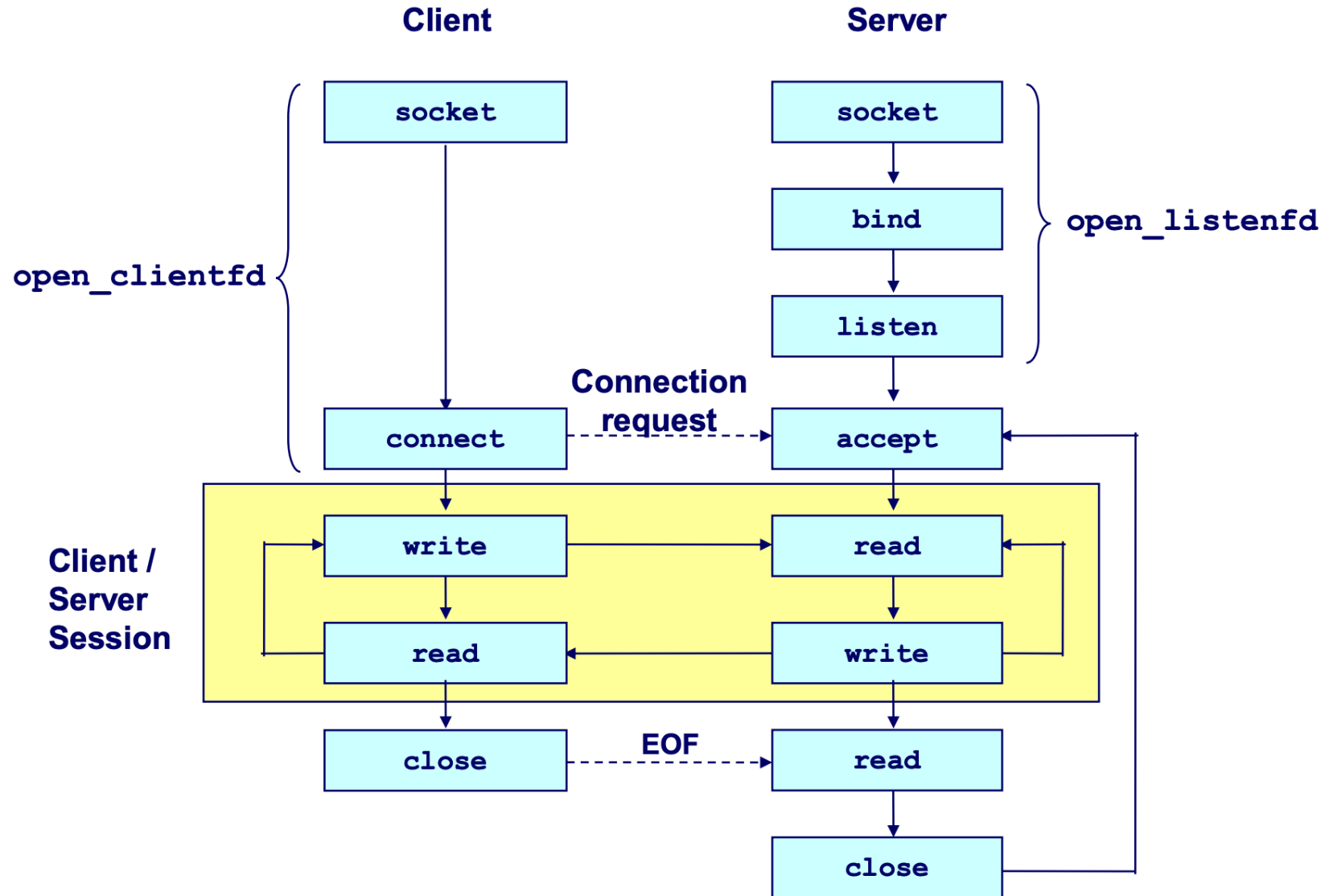


SCHOOL OF
DATA SCIENCE
數據科學學院

Recall socket programming



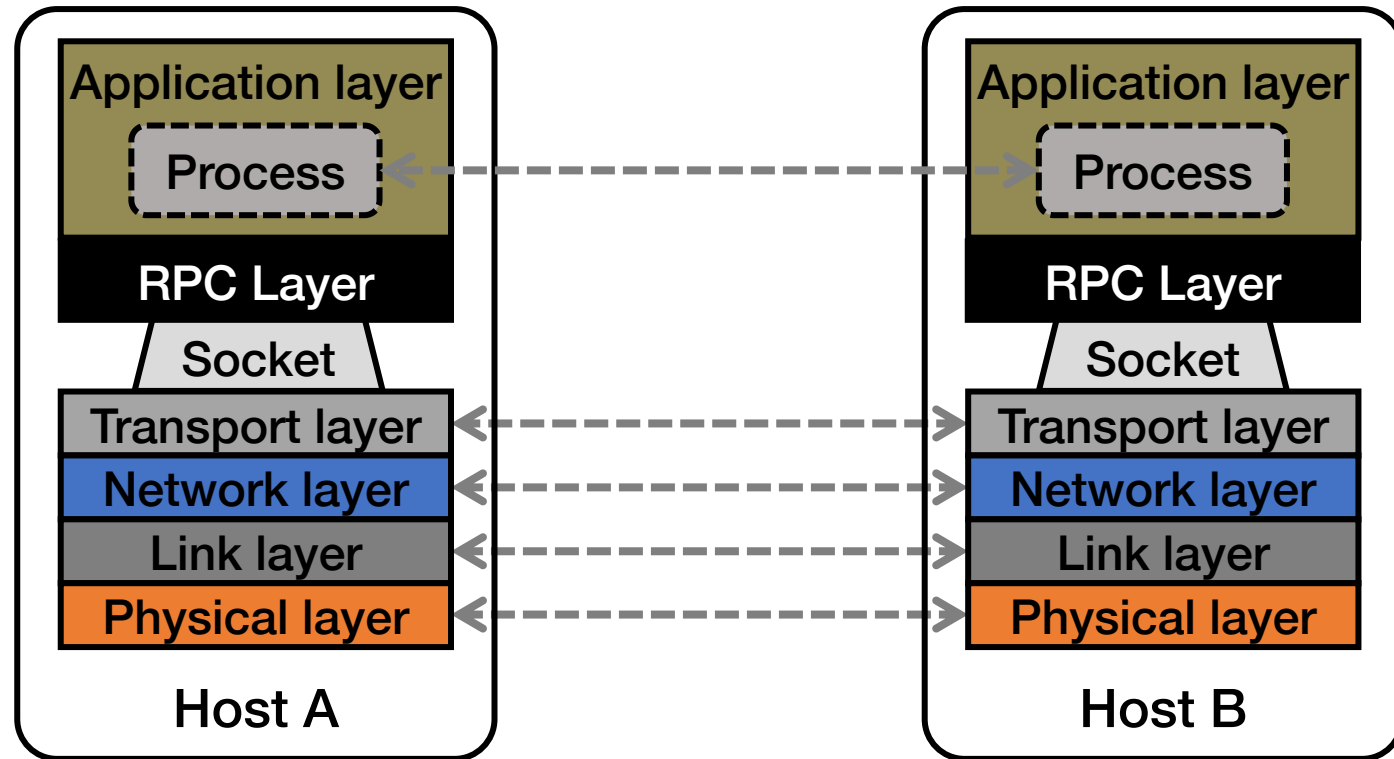
Recall socket programming



Socket works well enough?

- A lot of programming issues
 - How to separate different requests on the same connection?
 - How to write bytes to the network / read bytes from the network?
What if Host A's process is written in Python and Host B's process is in C++?
 - What to do with those bytes?

Solution: Remote Procedure Call (RPC)



Why RPC

- The typical programmer is trained to write single-threaded code that runs in one place
- Goal: Easy-to-program network communication that makes client-server communication transparent
 - Retains the “feel” of writing centralized code?
 - Programmer needn't think about the network

Many RPC implementations

- Google gRPC
- Facebook/Apache Thrift
- Twitter Finagle
- ...

Goals of RPC

- Within a single program, running in a single process, recall the well-known notion of a **procedure call**
 - **Caller** pushes arguments onto stack, jumps to address of callee function
 - **Callee** reads arguments from stack, executes, puts return value in register, returns to next instruction in caller

RPC's Goal: make communication appear like a local procedure call: way less painful than sockets...

RPC challenges

- Heterogeneity
 - Client needs to rendezvous with the server
 - Server must dispatch to the required function
 - What if server is different type of machine?
- Failure
 - What if messages get dropped?
 - What if client, server, or network fails?
- Performance
 - Procedure call takes ≈ 10 cycles ≈ 3 ns
 - RPC in a data center takes ≈ 10 μ s ($10^3\times$ slower)
 - In the wide area, typically $10^6\times$ slower

Problem: differences in data representation

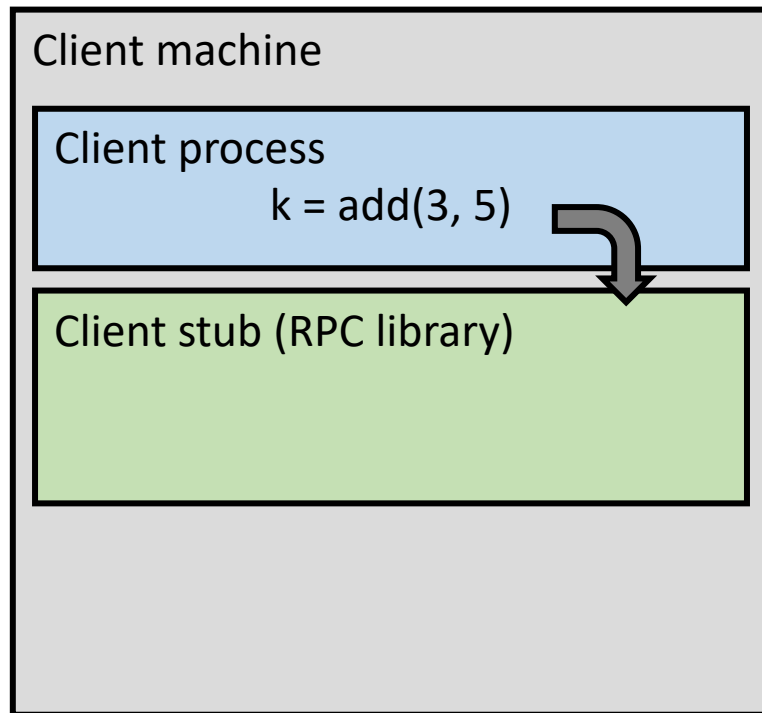
- Not an issue for local procedure calls
- For a remote procedure call, a remote machine may:
 - Run process written in a **different language**
 - Represent data types using **different sizes**
 - Use a **different byte ordering** (endianness)
 - Represent floating point numbers **differently**
 - Have **different data alignment** requirements
 - e.g., 4-byte type begins only on 4-byte memory boundary

Solution: Interface description language

- Mechanism to pass procedure parameters and return values in a machine-independent way
- Programmer may write an **interface description** in the IDL
 - Defines API for procedure calls: names, parameter/return types
- Then runs an **IDL compiler** which generates:
 - Code to **marshal** (convert) native data types into machine-independent byte streams
 - And vice-versa, called **unmarshaling**
 - Client stub: Forwards local procedure call as a request to server
 - Server stub: Dispatches RPC to its implementation

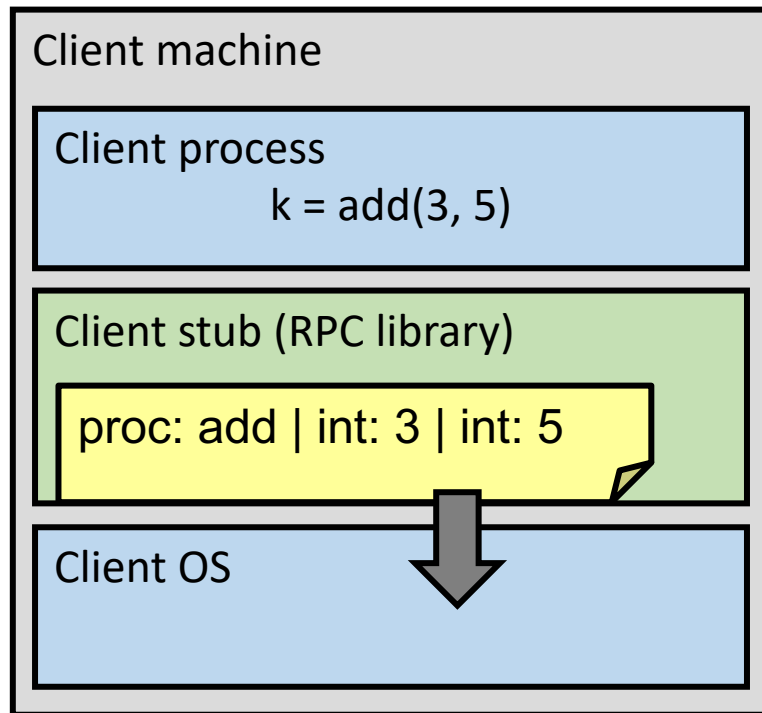
RPC

1. Client calls stub function (pushes parameters onto stack)



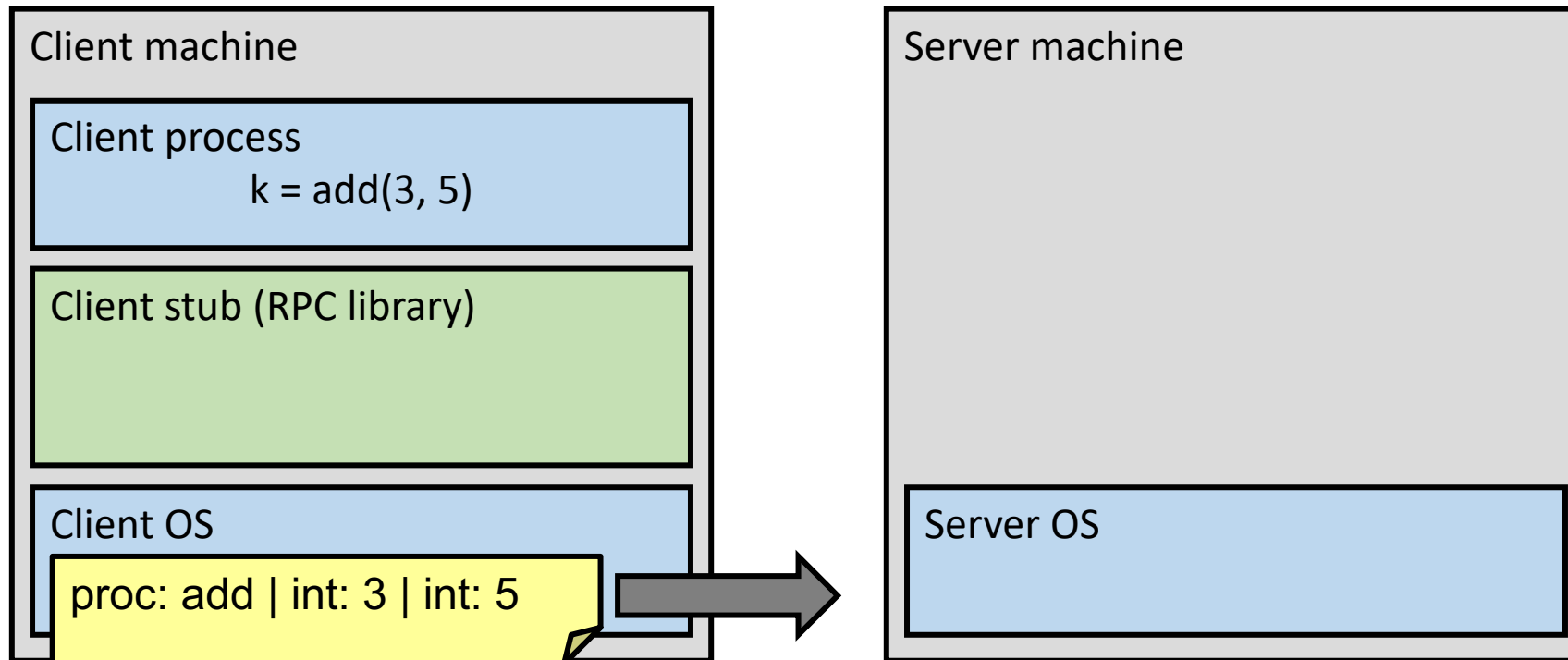
RPC

2. Stub marshals parameters to a network message



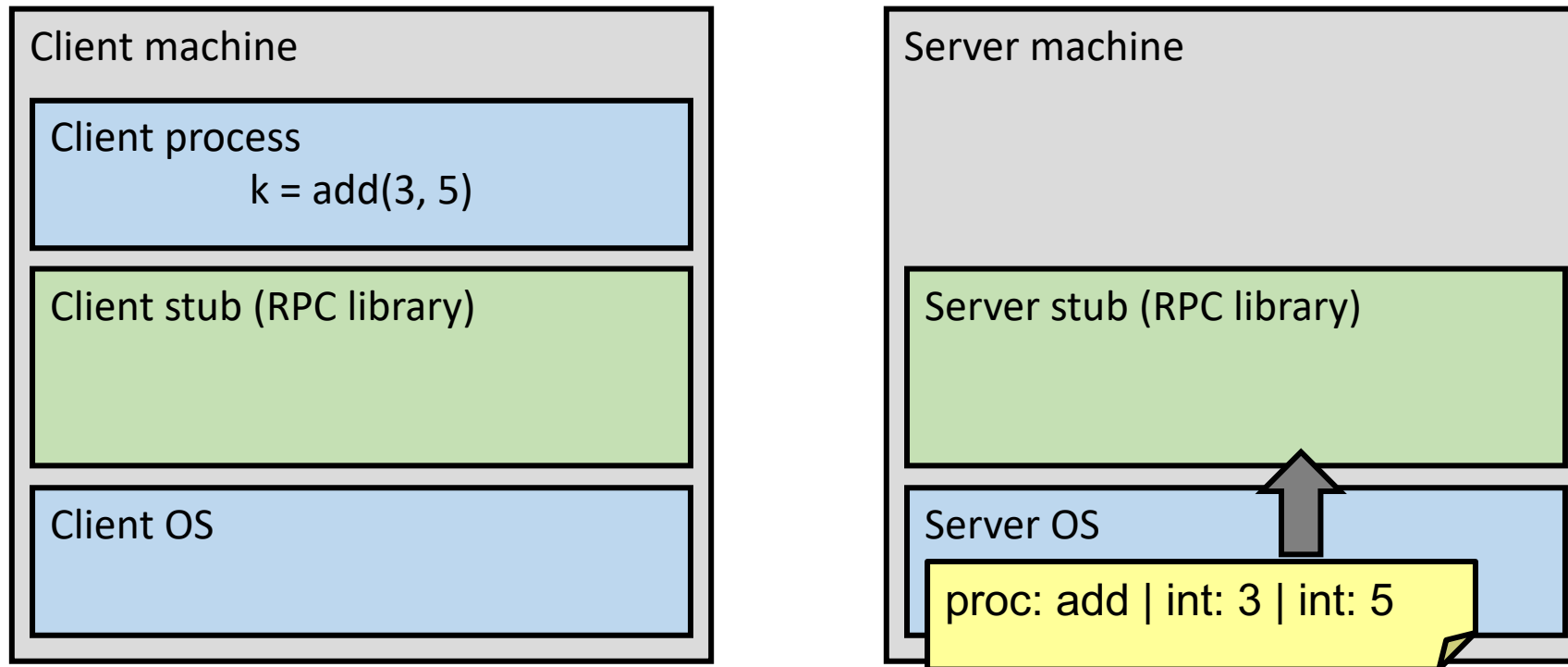
RPC

3. OS sends a network message to the server



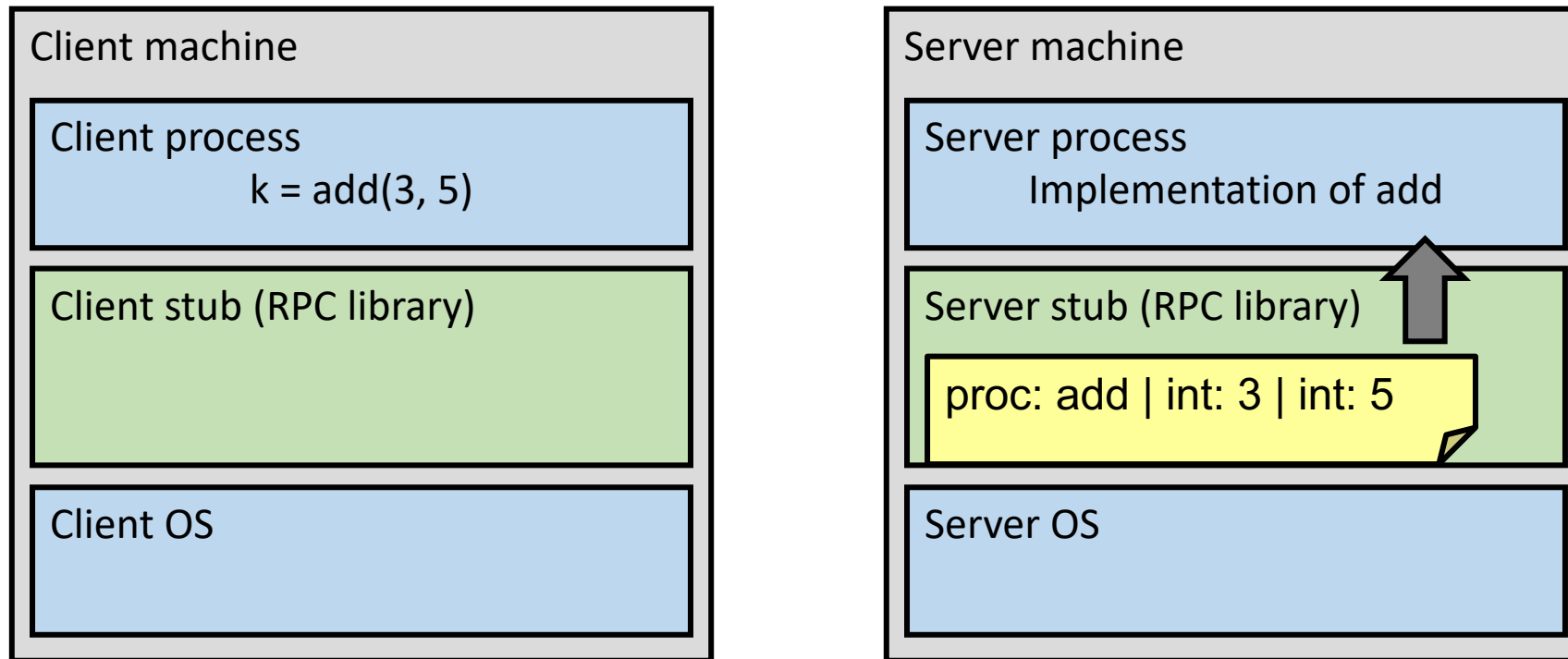
RPC

4. Server OS receives message, sends it up to stub



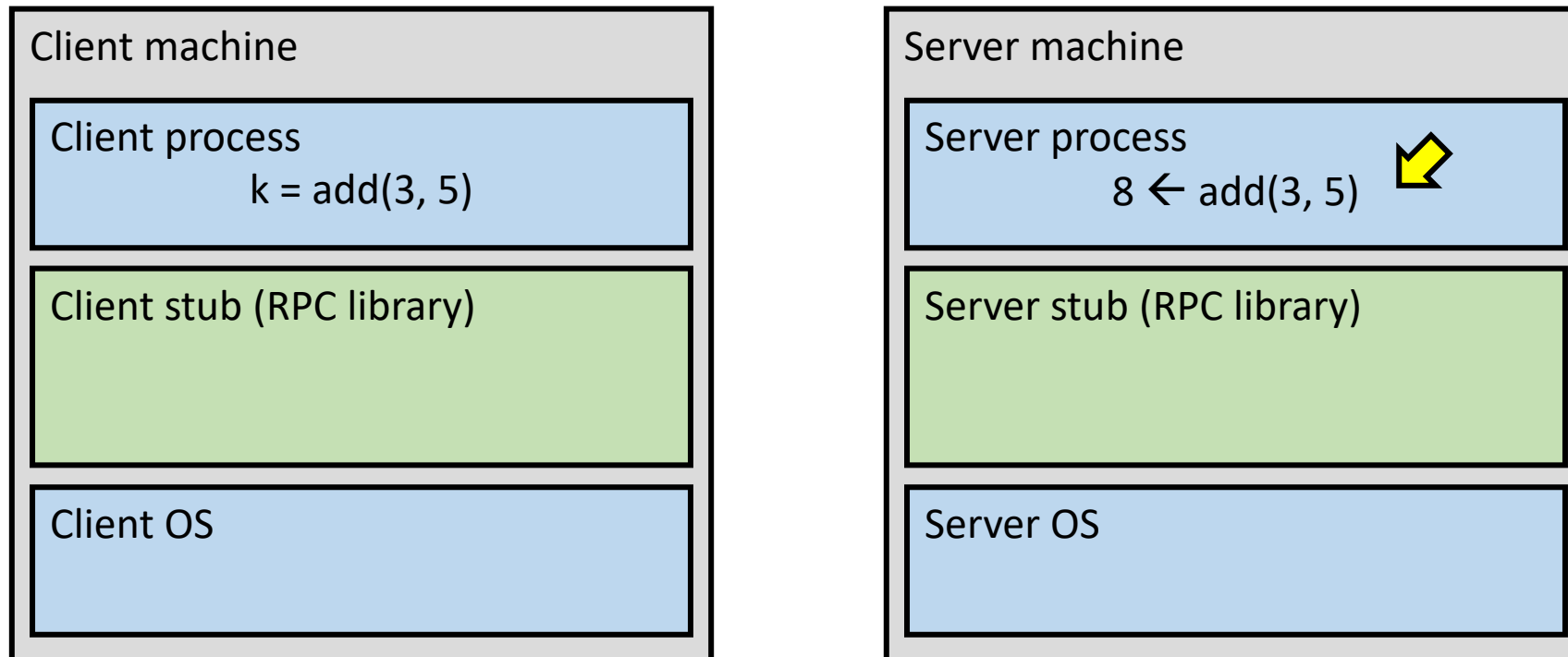
RPC

5. Server stub unmarshals params, calls server function



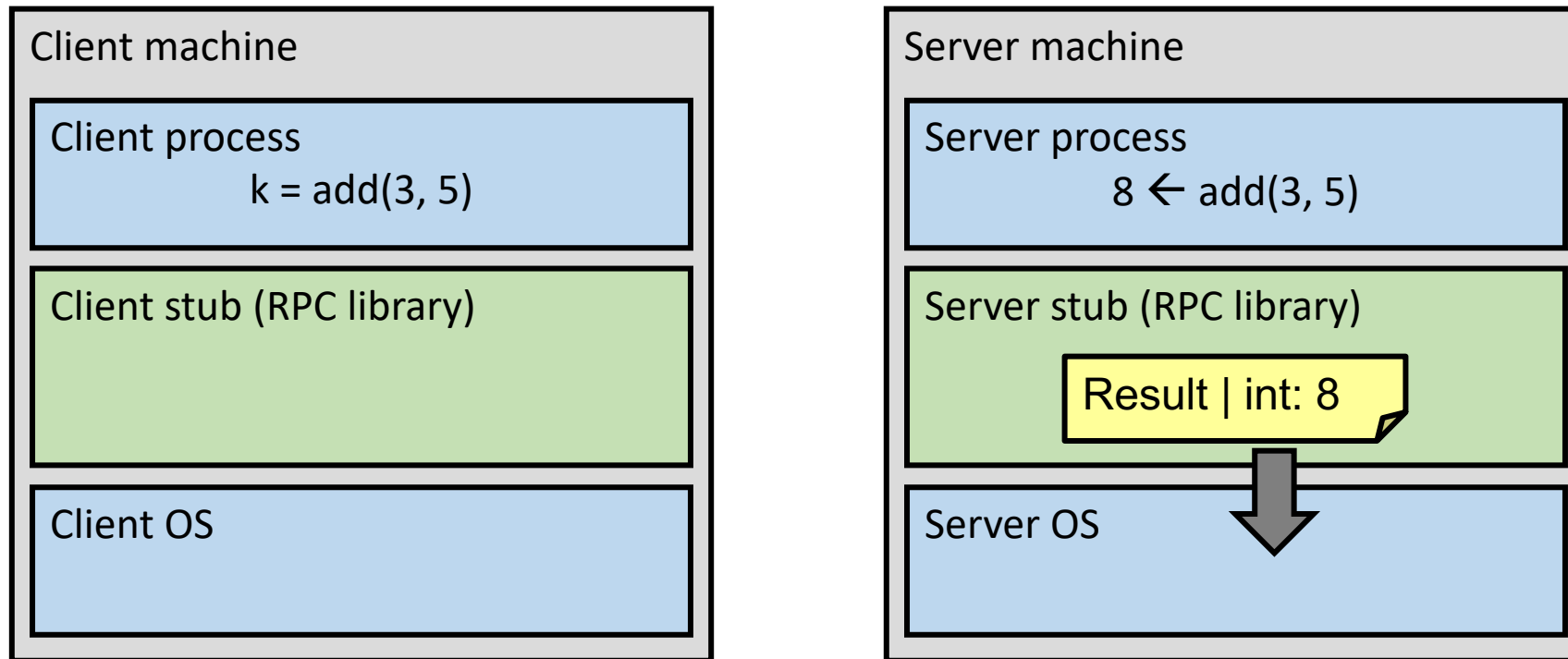
RPC

6. Server function runs, returns a value



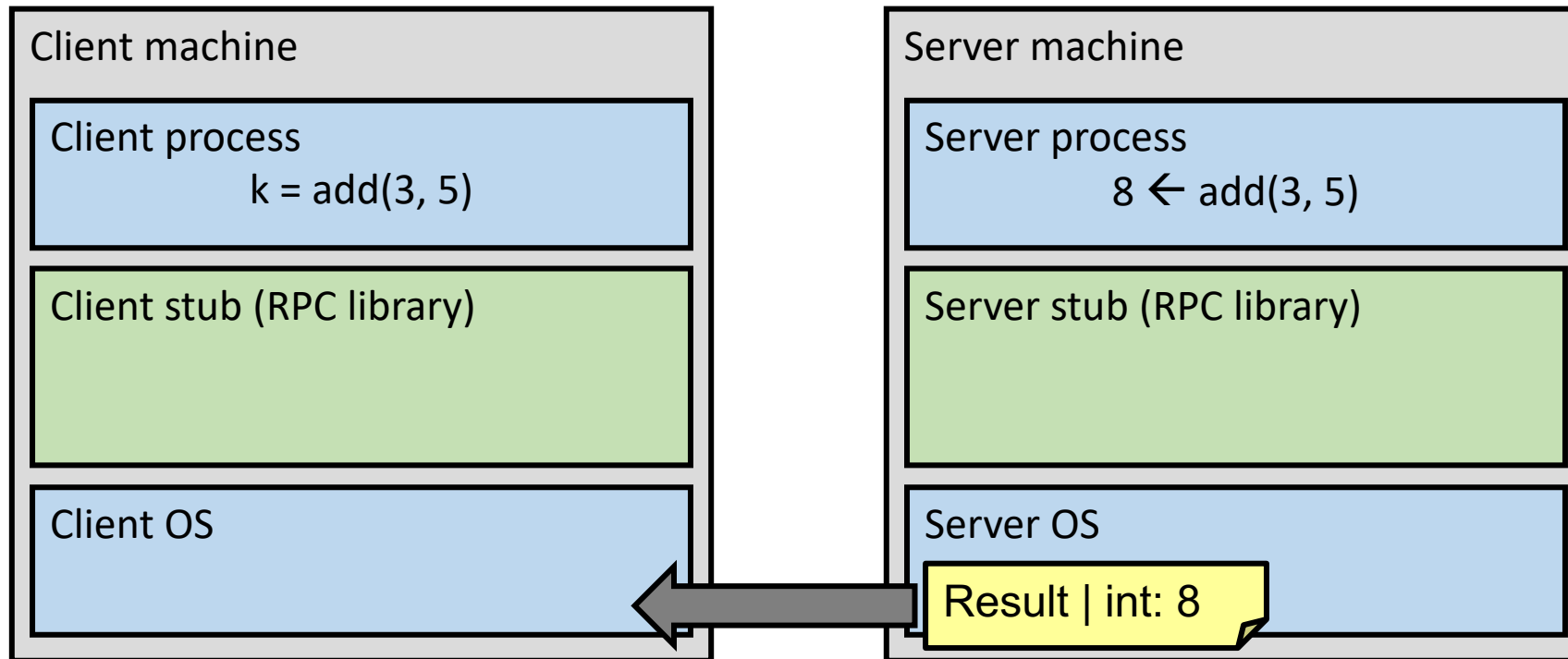
RPC

7. Server stub marshals the return value, sends message



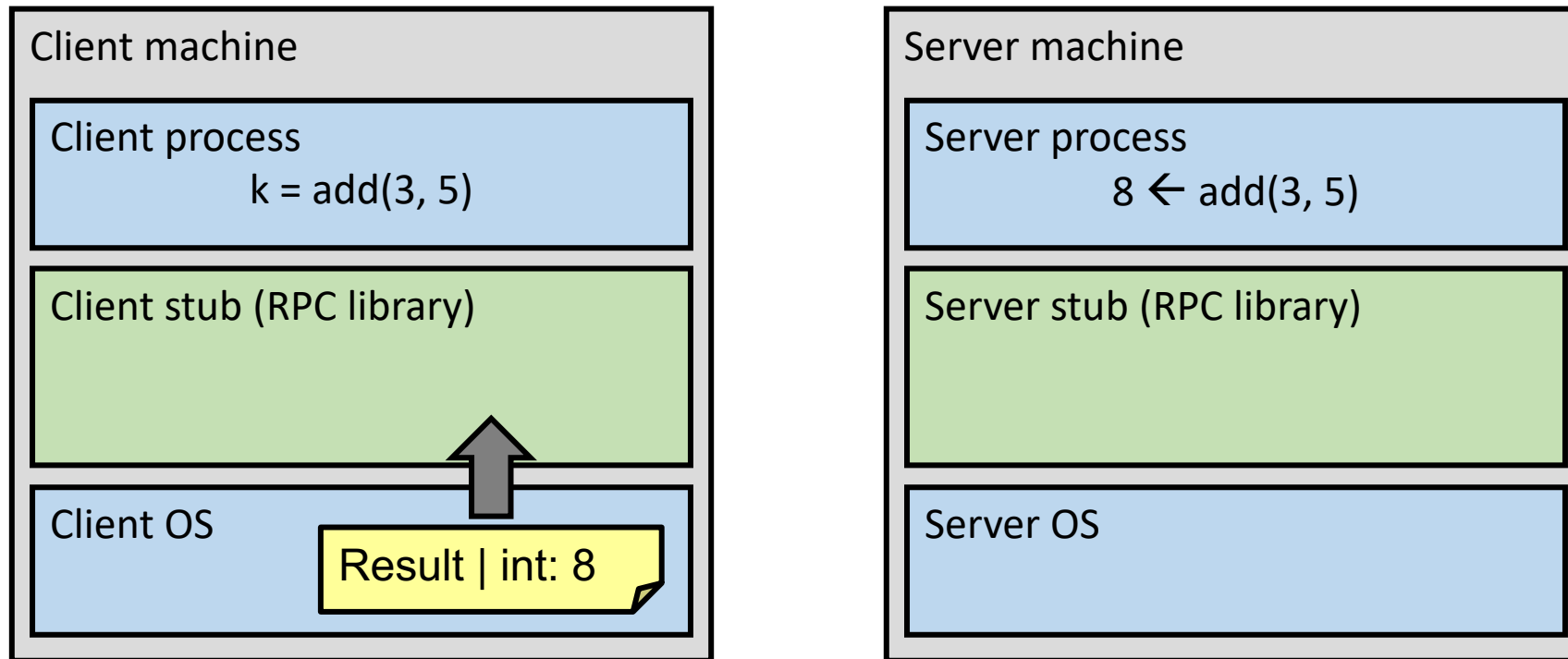
RPC

8. Server OS sends the reply back across the network



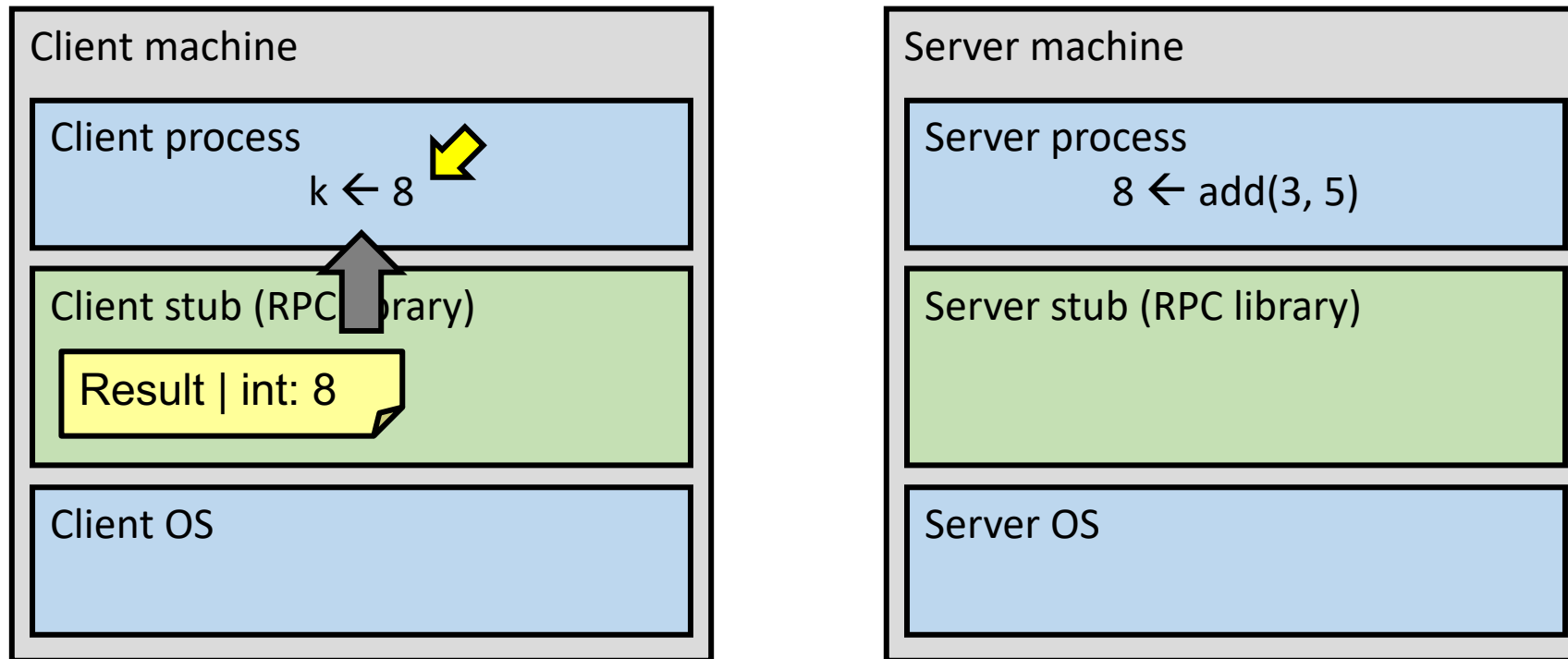
RPC

9. Client OS receives the reply and passes up to stub



RPC

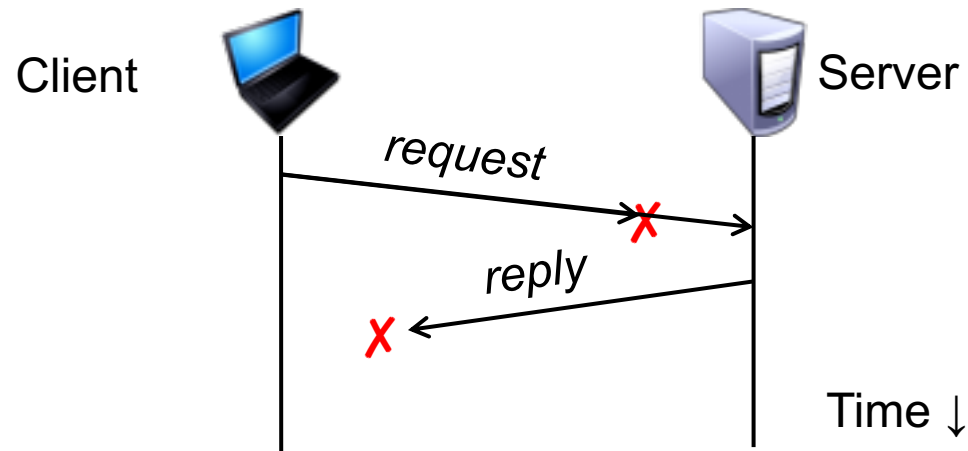
10. Client stub unmarshals return value, returns to client



What could possibly go wrong?

- Client may **crash and reboot**
- Packets may be **dropped**
 - Some individual packet loss in the Internet
 - Broken routing results in many lost packets
- Server may **crash and reboot**
- Network or server might just be **very slow**

Failures, from client's perspective



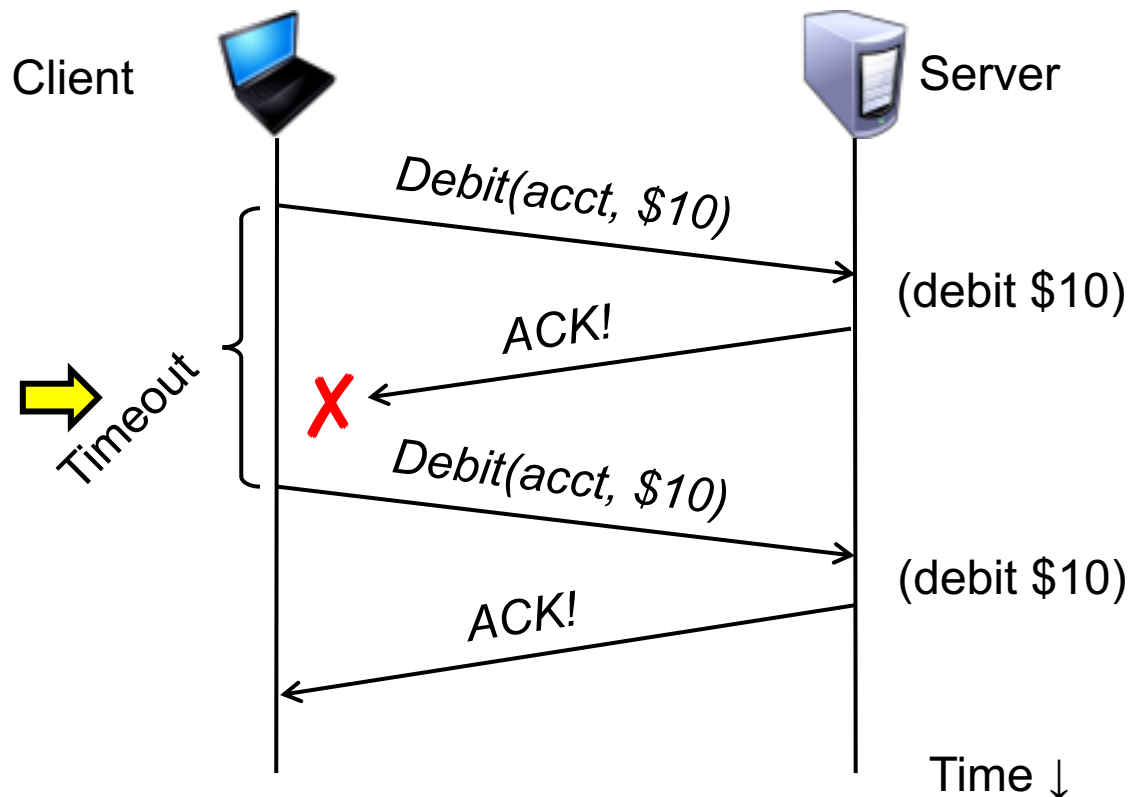
The cause of the failure is **hidden** from the **client**!

At-Least-Once scheme

- Simplest scheme for handling failures
 1. Client stub waits for a response, for a while
 - Response is an **acknowledgement** message from the server stub
 2. If no response arrives after a fixed **timeout** time period, then client stub re-sends the request
- Repeat the above a few times
 - Still no response? Return an error to the application

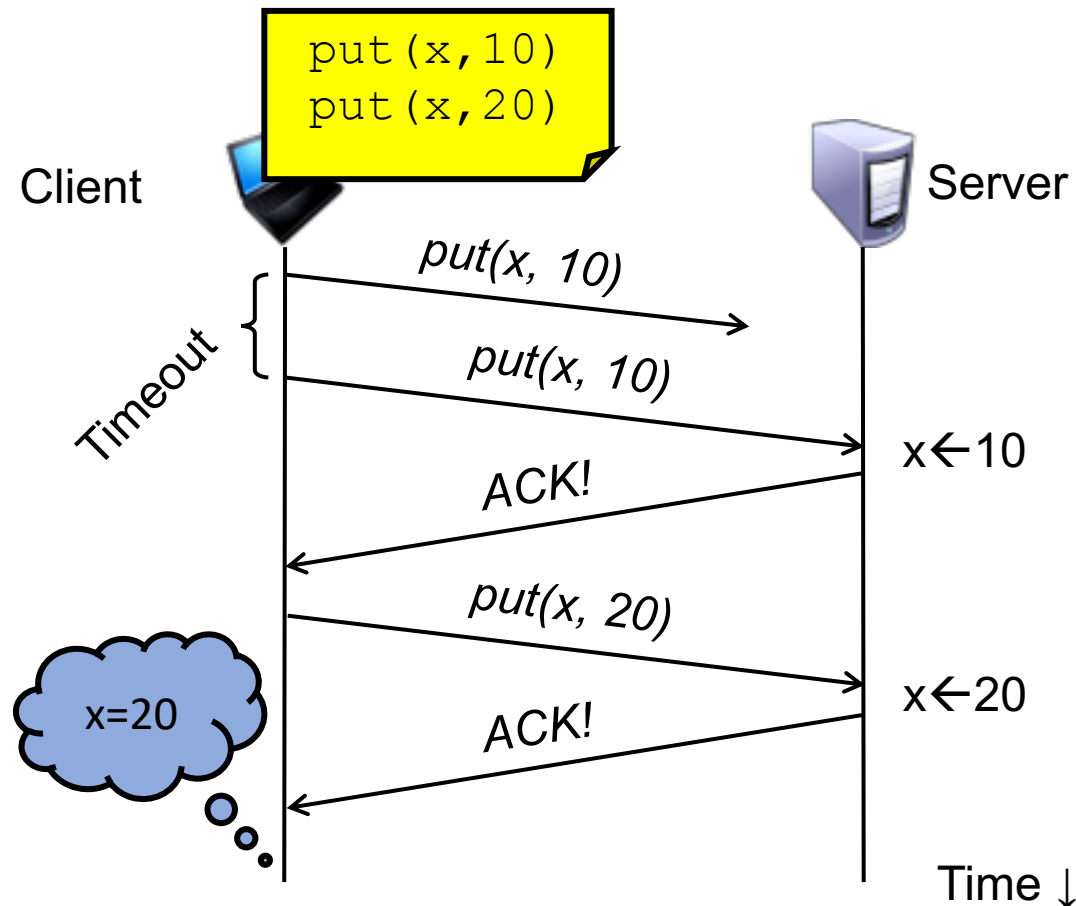
At-Least-Once and side effects

- Client sends a “debit \$10 from bank account” RPC



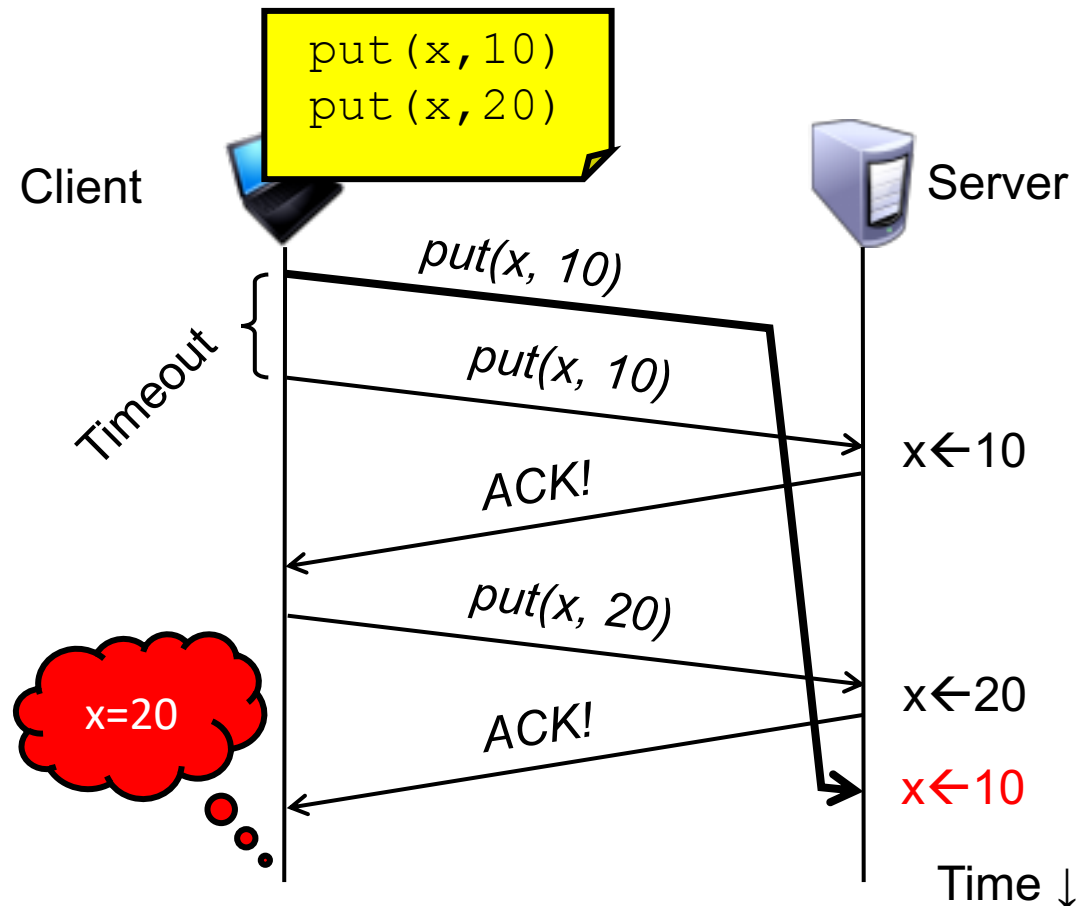
At-Least-Once and writes

- `put(x, value)`, then `get(x)`: expect answer to be value



At-Least-Once and writes

- Consider a client storing key-value pairs in a database
 - `put(x, value)`, then `get(x)`: expect answer to be value



So is At-Least-Once ever okay?

- Yes: If they are read-only operations with no side effects
 - e.g., read a key's value in a database
- Yes: If the application has its own functionality to cope with duplication and reordering

At-Most-Once scheme

- Idea: server RPC stub detects duplicate requests
 - Returns previous reply instead of re-running handler
- How to detect a duplicate request?
 - Test: Server stub sees same function, same arguments twice
 - **No!** Sometimes applications legitimately submit the same function with same arguments, twice in a row

At-Most-Once scheme

- How to detect a duplicate request?
 - Client stub includes unique **transaction ID** (xid) with each RPC request
 - Client stub uses same xid for retransmitted requests

```
At-Most-Once Server Stub  
if seen[xid]:  
    retval = old[xid]  
else:  
    retval = handler()  
    old[xid] = retval  
    seen[xid] = true  
return retval
```

At-Most-Once: Providing unique XIDs

- Combine a unique client ID (e.g., IP address) with the current time of day
- Combine unique client ID with a sequence number
- Big random number (probabilistic, not certain guarantee)

At-Most-Once: Discarding server state

- Problem: seen and old arrays will grow without bound
- Observation: By construction, when the client gets a response to a particular xid, it will never re-send it
- Client could tell server “I’m done with xid x – delete it”
 - Have to tell the server about each and every retired xid
 - Could piggyback on subsequent requests

Significant overhead if many RPCs are in flight, in parallel

At-Most-Once: Discarding server state

- Suppose $xid = \langle \text{unique client id, sequence no.} \rangle$
 - e.g., $\langle 42, 1000 \rangle$, $\langle 42, 1001 \rangle$, $\langle 42, 1002 \rangle$
- Client includes “seen all replies $\leq X$ ” with every RPC
 - Much like TCP sequence numbers, acks
- How does the client know that the server received the information about retired RPCs?
 - Each one of these is cumulative: later seen messages subsume earlier ones

At-Most-Once: Concurrent requests

- Problem: How to handle a duplicate request while the original is still executing?
 - Server doesn't know reply yet. Also, we don't want to run the procedure twice
- Idea: Add a **pending** flag per executing RPC
 - Server waits for the procedure to finish, or ignores

At-Most-Once: Server crash and restart

- Problem: Server may crash and restart
- Does server need to write its tables to disk?
- Yes! On server crash and restart:
 - If `old[]`, `seen[]` tables are only in memory:
 - Server will forget, accept duplicate requests

Exactly-once?

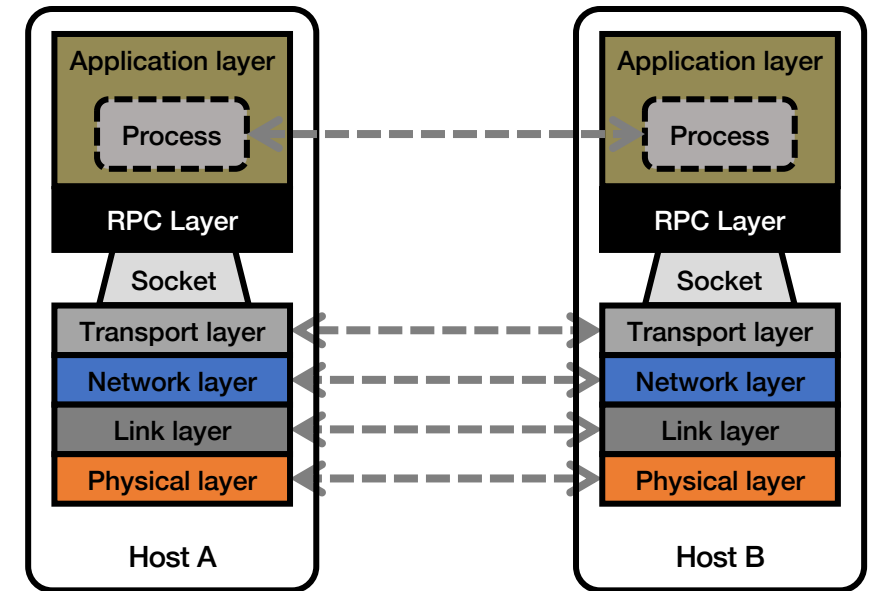
- Need retransmission of at least once scheme
- Plus the duplicate filtering of at most once scheme
 - To survive client crashes, client needs to record pending RPCs on disk
 - So it can replay them with the same unique identifier
- Plus story for making server reliable
 - Even if server fails, it needs to continue with full state
 - To survive server crashes, server should log to disk results of completed RPCs (to suppress duplicates)

Exactly-once for external actions?

- Imagine that the remote operation triggers an external physical thing
 - e.g., dispense \$100 from an ATM
- The ATM could crash immediately before or after dispensing and lose its state
 - Don't know which one happened
 - Operation is not atomic (logging and dispensing are separate actions)
- So **can't achieve exactly-once in general**, in the presence of external actions

Summary: RPCs and Network Comm.

- Layers are our friends!
- RPCs are everywhere
- Necessary issues surrounding machine heterogeneity
- Subtle issues around failures
 - At-least-once w/ retransmission
 - At-most-once w/ duplicate filtering
 - Discard server state w/ cumulative acks
 - Exactly-once with:
 - at-least-once + at-most-once
+ fault tolerance + no external actions



Credits

- Some slides are adapted from course slides of COS 418/518 in Princeton